Regularisation and model validation

Machine Learning, lecture 3

Makhmudjon Sodikov
YMLS Uzbekistan, fall 2026

YMLS girafe ai

Two loose ends from last week

Both were left for later.

−6.4 to 5.2

The weight of a twin feature, depending on the sample. Which one do we report?

136.5

The lowest MSE on our 40 flats. Is that the best model for the next flat?

Today

1. When a better fit is a worse model
2. Regularisation
3. Hyperparameters and validation
4. Splitting the data right

When a better fit is a worse model

The features are ours to choose

Linear in the weights, as last week.

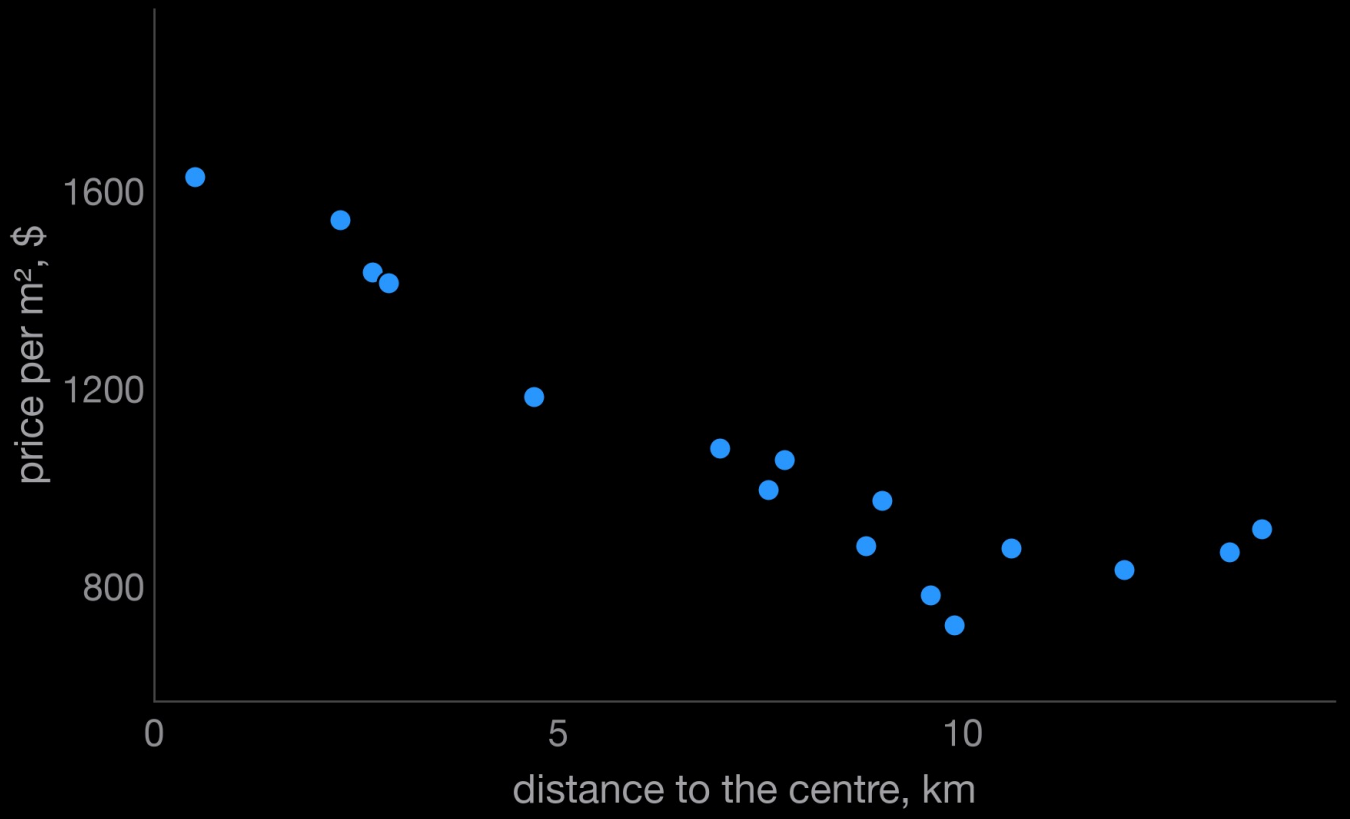
$$x \longmapsto (x,\ x^2,\ \ldots,\ x^d)$$

$$\hat{y} = w_0 + w_1 x + w_2 x^2 + \cdots + w_d x^d$$

A polynomial of degree *d* is a linear model with *d* features.

Least squares finds the weights. Nobody has told us *d*.

Sixteen flats

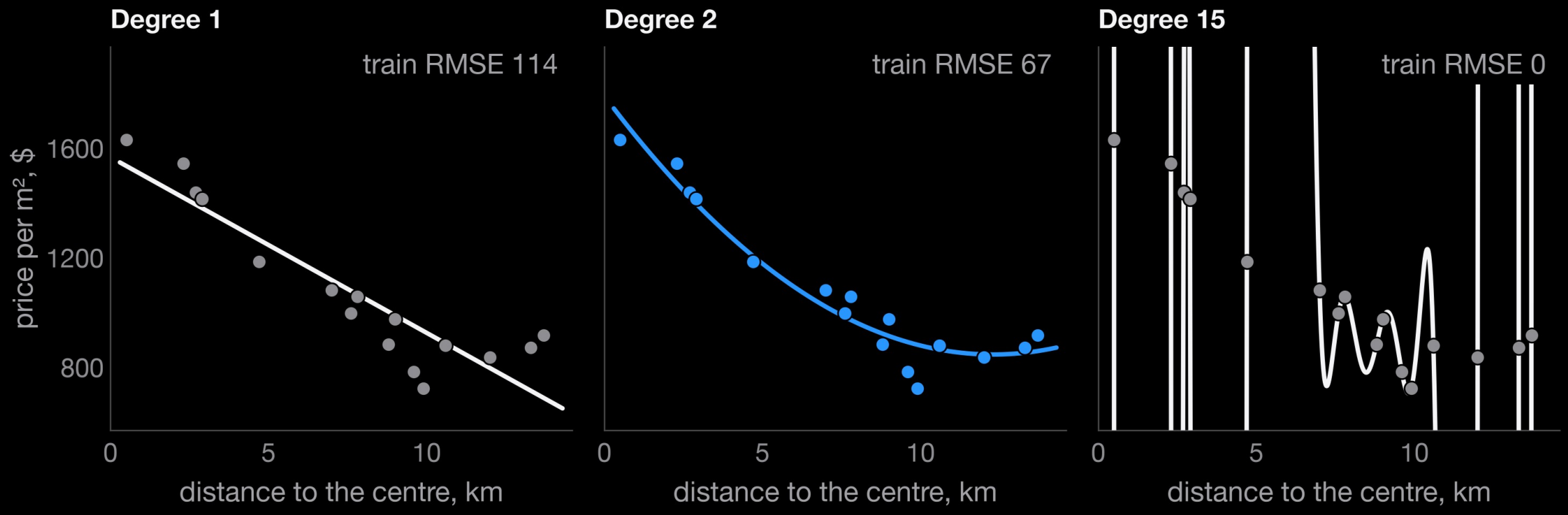


16

flats

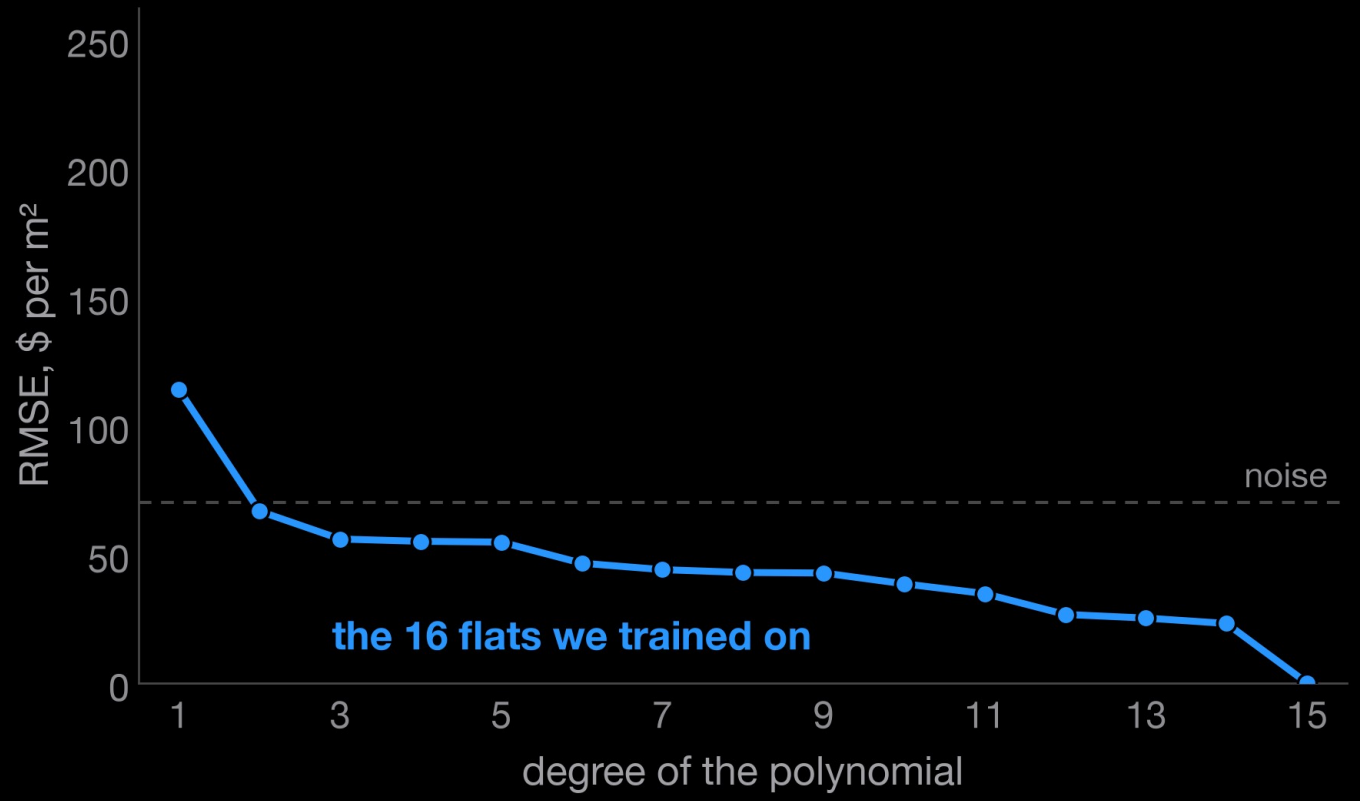
Price per square metre against the distance to the centre.

Three models, one dataset



A line misses the bend. Degree 15 passes through every flat.

A higher degree always fits better



Training error can only fall.

Every extra feature gives least squares one more way to follow the points.

With 16 weights for 16 flats it follows all of them.

The same model, two errors

The degree 15 polynomial.

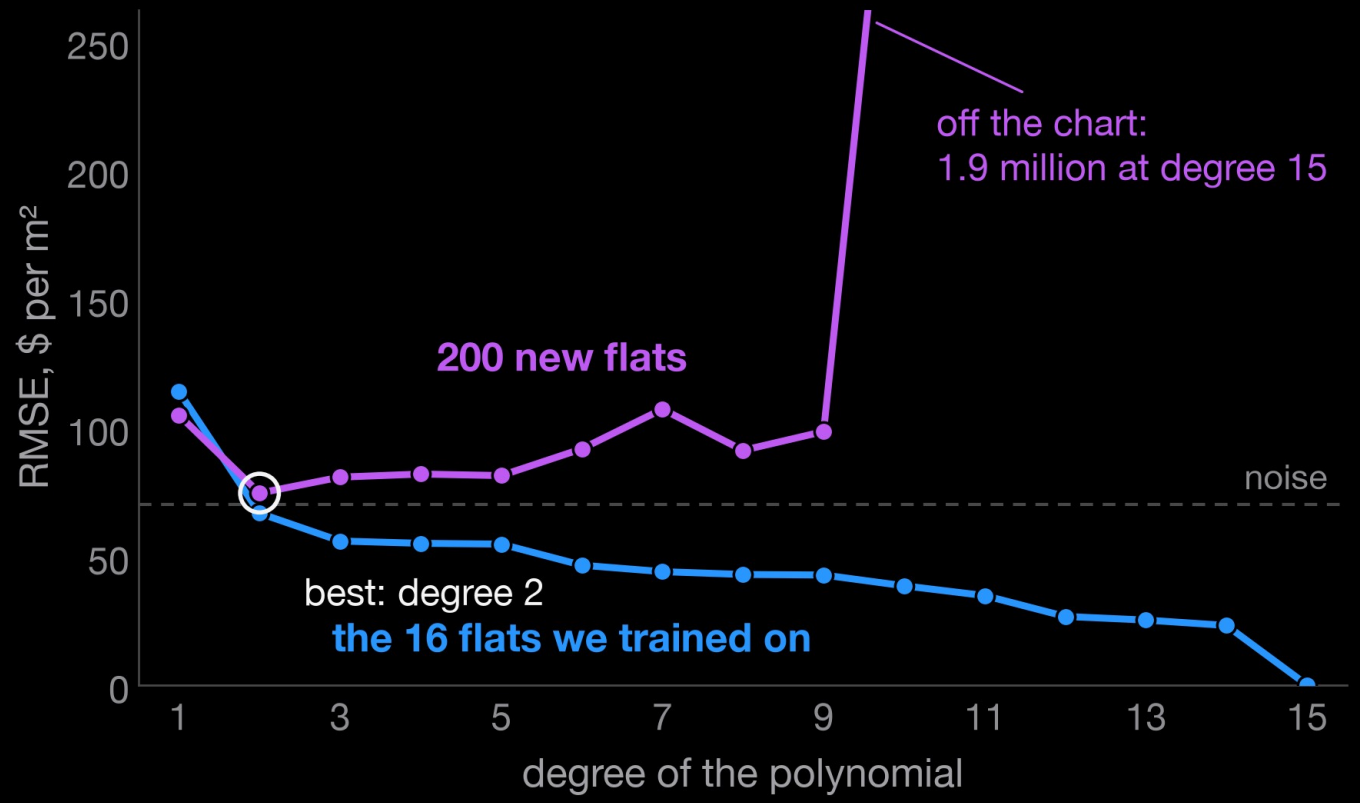
0

RMSE on the 16 flats it was trained on.

1 927 955

RMSE on 200 new flats, in dollars per square metre.

The error on new data turns back up



Lowest at degree 2, then it climbs.

We want the purple curve to be low.

Training only ever sees the blue one.

Underfitting and overfitting

Underfitting

The model is too simple for the data.

High error on the training set and on new data.

Degree 1: it cannot bend.

Overfitting

The model follows the noise of its training set.

Low training error, high error on new data.

Degree 15: it bends everywhere.

Capacity: how much a model can fit

The degree, the number of features, *k* in kNN: each sets the capacity.

Too little

Degree 1. Misses the pattern. More data will not help.

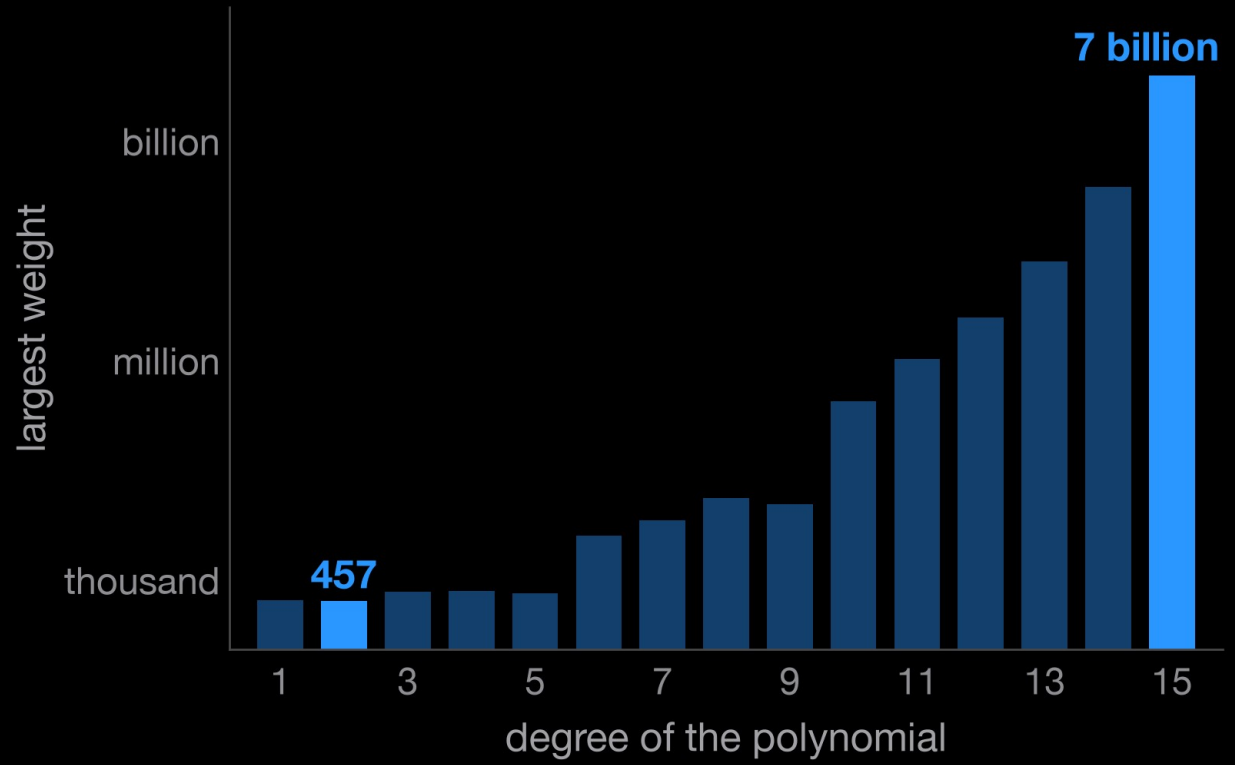
About right

Degree 2. Catches the pattern and leaves the noise alone.

Too much

Degree 15. Fits anything, the noise included.

The weights give it away

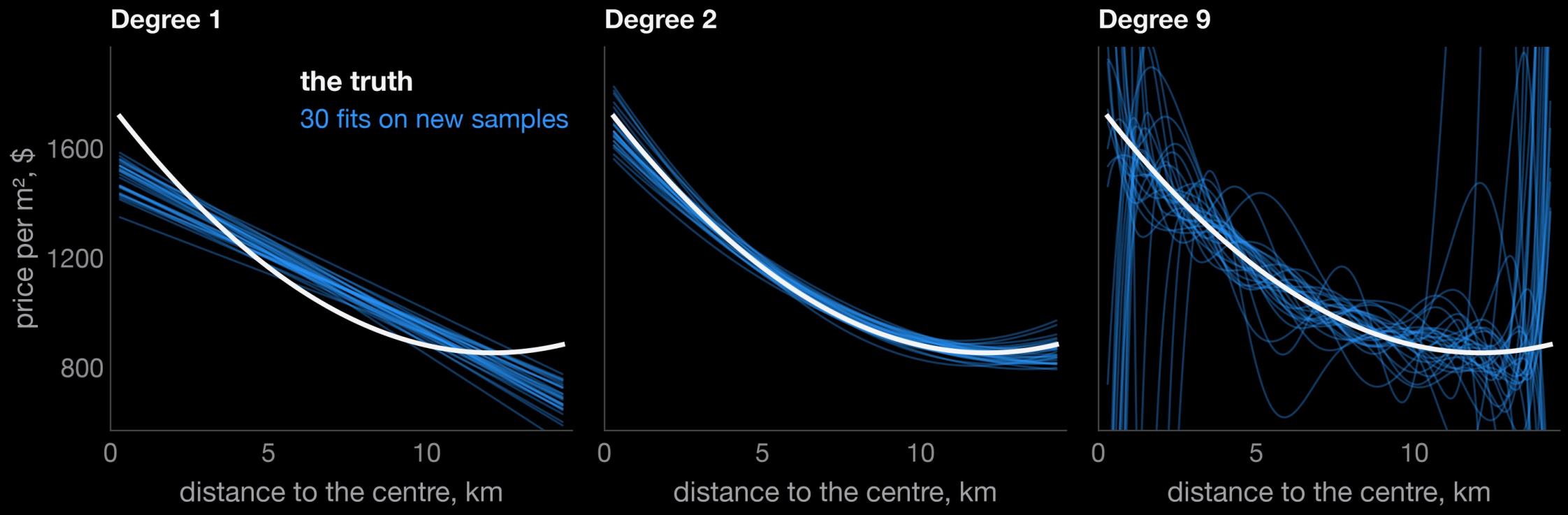


An overfitted model has huge weights.

Degree 2 needs weights of a few hundred.

Degree 15 needs billions, with both signs, cancelling on the 16 flats and nowhere else.

Train again on new flats



High bias

steady, and wrong

High variance

a new curve every time

The error splits into three parts

The expected squared error at a new point, over training samples and noise.

$$y = f(x) + \varepsilon, \qquad \mathbb{E}\,\varepsilon = 0, \quad \operatorname{Var}\varepsilon = \sigma^2$$

$$\mathbb{E}\Big[\big(y - \hat{f}(x)\big)^2\Big] = \underbrace{\sigma^2}_{\text{noise}} + \underbrace{\big(f(x) - \mathbb{E}\,\hat{f}(x)\big)^2}_{\text{bias}^2} + \underbrace{\operatorname{Var}\hat{f}(x)}_{\text{variance}}$$

Noise

No model can remove it.

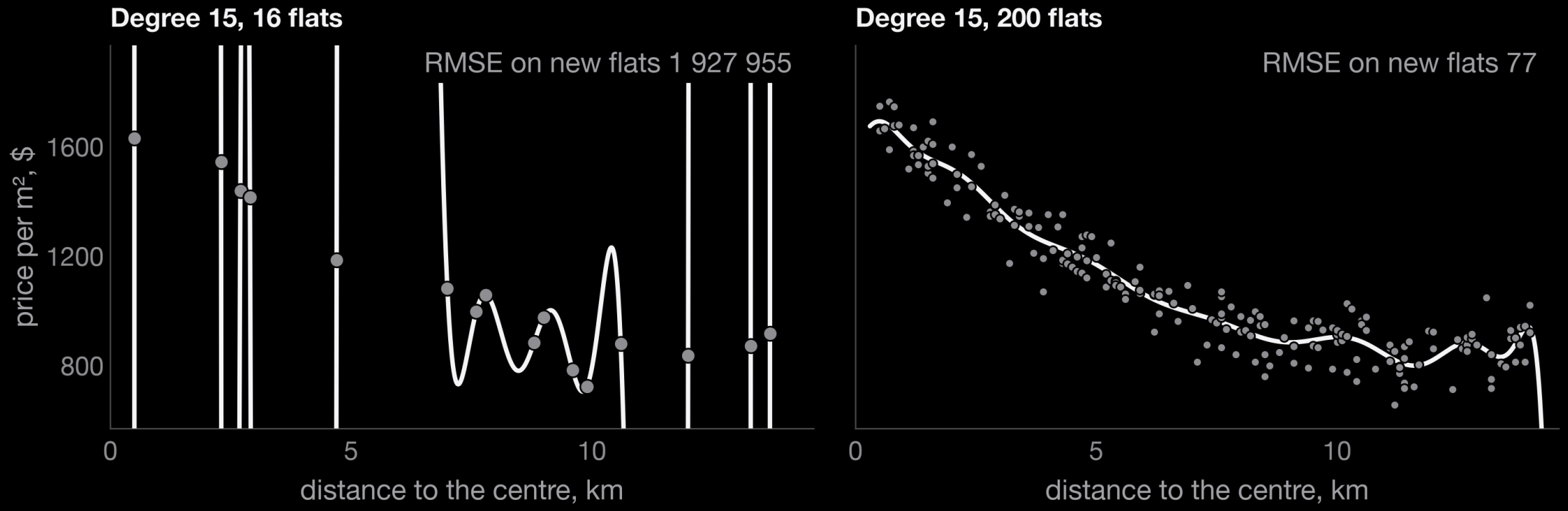
Bias

The average fit misses the truth. Falls as capacity grows.

Variance

The fit changes with the sample. Grows with capacity, falls with more data.

More data helps



The same degree 15. Two hundred flats leave it no room to swing.

Three ways out

Simpler model

A lower degree, fewer features. We have to know which ones to drop.

More data

Works every time.
Usually we cannot get it.

Regularisation

Keep the flexible model and stop its weights from growing.

Regularisation

Penalise large weights

$$L(w) = \underbrace{\|y - Xw\|^2}_{\text{fit the data}} + \lambda \underbrace{R(w)}_{\text{keep the weights small}}$$

λ is the price of a large weight. At $\lambda = 0$ we are back to least squares.

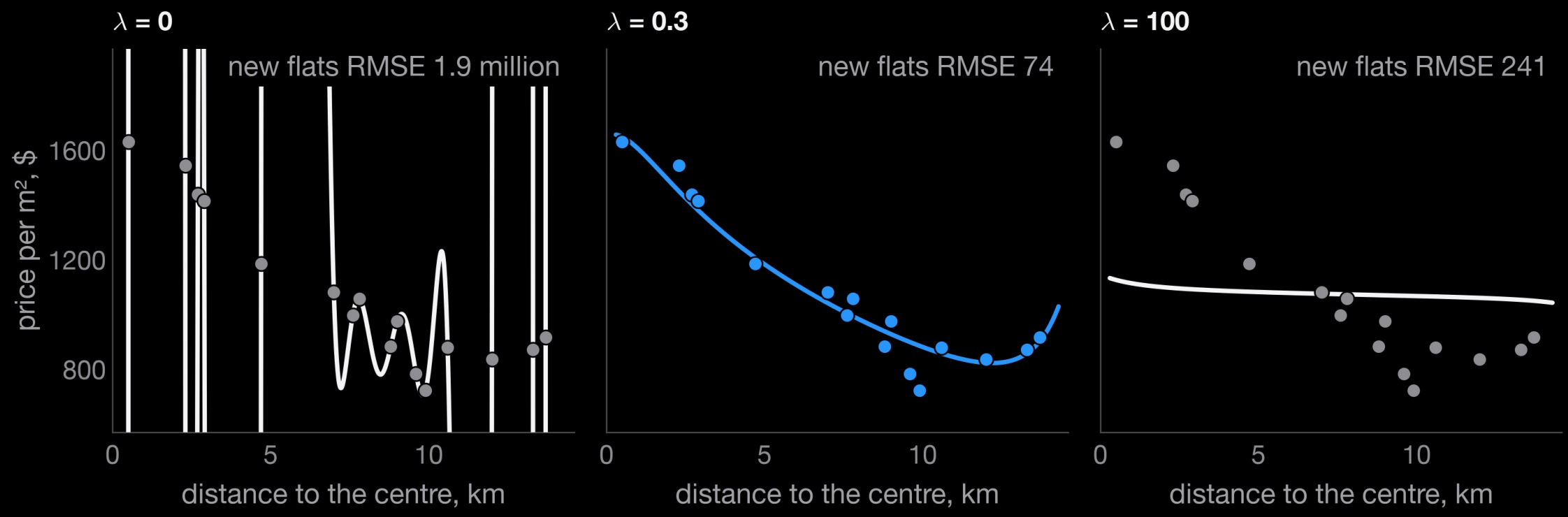
Ridge regression: the L2 penalty

$$L(w) = \|y - Xw\|^2 + \lambda \|w\|_2^2$$

$\|w\|_2^2 = \sum_{j=1}^{p} w_j^2$ the sum of squared weights

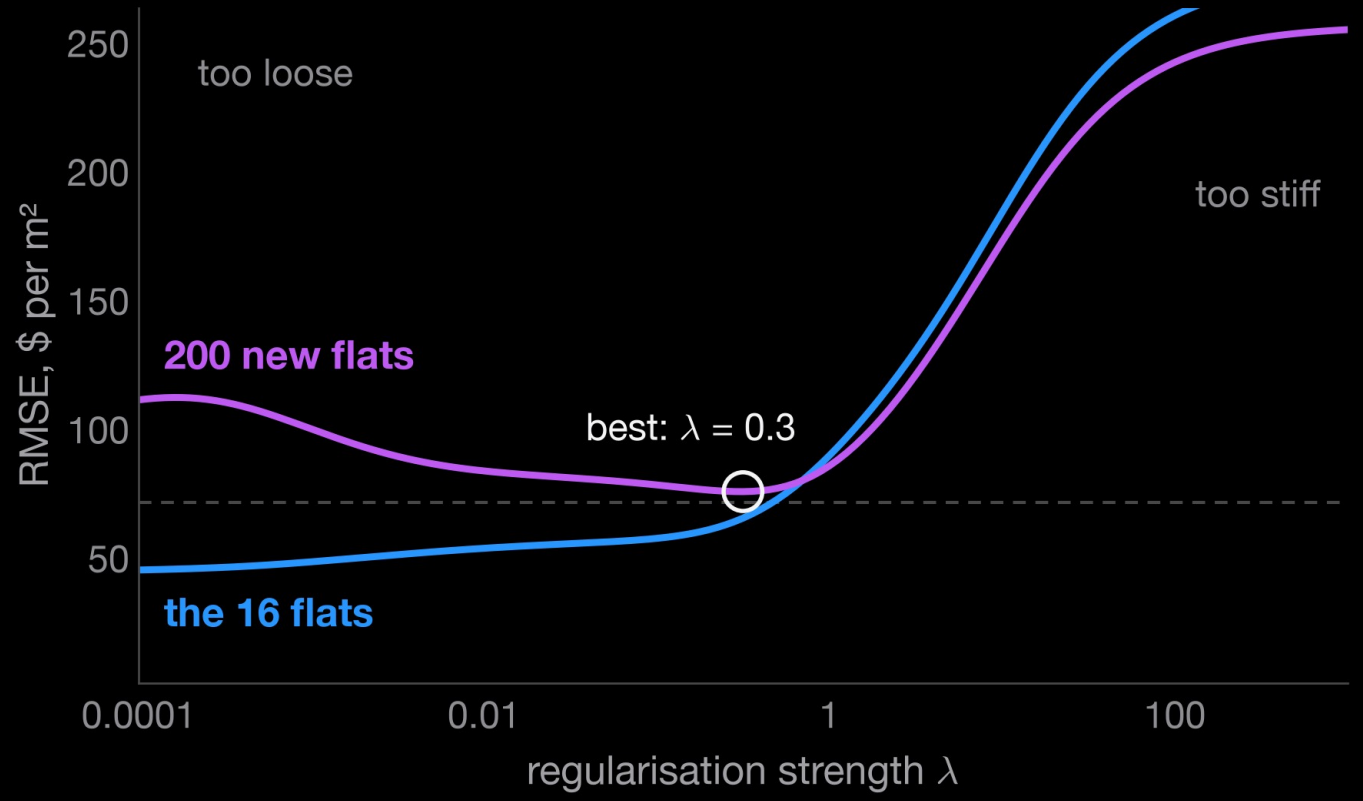
Also called Tikhonov regularisation.

One dial, three models



Still degree 15. λ moves it from overfitting to underfitting.

The same U, now along λ



The best λ is in the middle.

At λ = 0.3 the degree 15 model has RMSE 74 on new flats. The parabola had 74.5.

Capacity is now a continuous dial.

Ridge has a formula

Set the gradient to zero, as last week.

$$\hat{w} = (X^\top X)^{-1} X^\top y$$ least squares

$$\hat{w} = (X^\top X + \lambda I)^{-1} X^\top y$$ **ridge**

With λ on the diagonal the matrix can always be inverted, twins or not.

Where the ridge formula comes from

Differentiate, set the gradient to zero, solve for the weights.

$$L(w) = (y - Xw)^\top (y - Xw) + \lambda\, w^\top w$$

$$\nabla L(w) = -2X^\top (y - Xw) + 2\lambda w = 0$$

$$(X^\top X + \lambda I)\, w = X^\top y$$

$$\hat{w} = (X^\top X + \lambda I)^{-1} X^\top y$$

$$\nabla^2 L(w) = 2\,(X^\top X + \lambda I) \succ 0 \quad \text{for } \lambda > 0$$

The loss is strictly convex, so this point is its only minimum.

What λ does to the matrix

Split the data into its directions with the singular value decomposition.

$$X = UDV^\top \quad \Longrightarrow \quad X^\top X = VD^2V^\top, \qquad D = \operatorname{diag}(d_1, \ldots, d_p)$$

$$X^\top X + \lambda I = V\left(D^2 + \lambda I\right)V^\top, \qquad \text{eigenvalues} \;\; d_j^2 + \lambda \;\geq\; \lambda \;>\; 0$$

Every eigenvalue moves up by λ. None of them is zero, so the inverse exists.

$$X\hat{w} = \sum_{j=1}^{p} u_j \, \frac{d_j^2}{d_j^2 + \lambda} \, u_j^\top y$$

Ridge shrinks each direction of the data by its own factor. Weak directions, like the difference of the twins, shrink the most.

$$\operatorname{df}(\lambda) = \sum_{j=1}^{p} \frac{d_j^2}{d_j^2 + \lambda}$$

The effective number of parameters. It equals p without a penalty and falls as λ grows.

Ridge trades bias for variance

A linear signal plus independent noise, as last week.

least squares

$$\mathbb{E}\,\hat{w} = w, \qquad \mathrm{Var}\left(v_j^\top \hat{w}\right) = \frac{\sigma^2}{d_j^2}$$

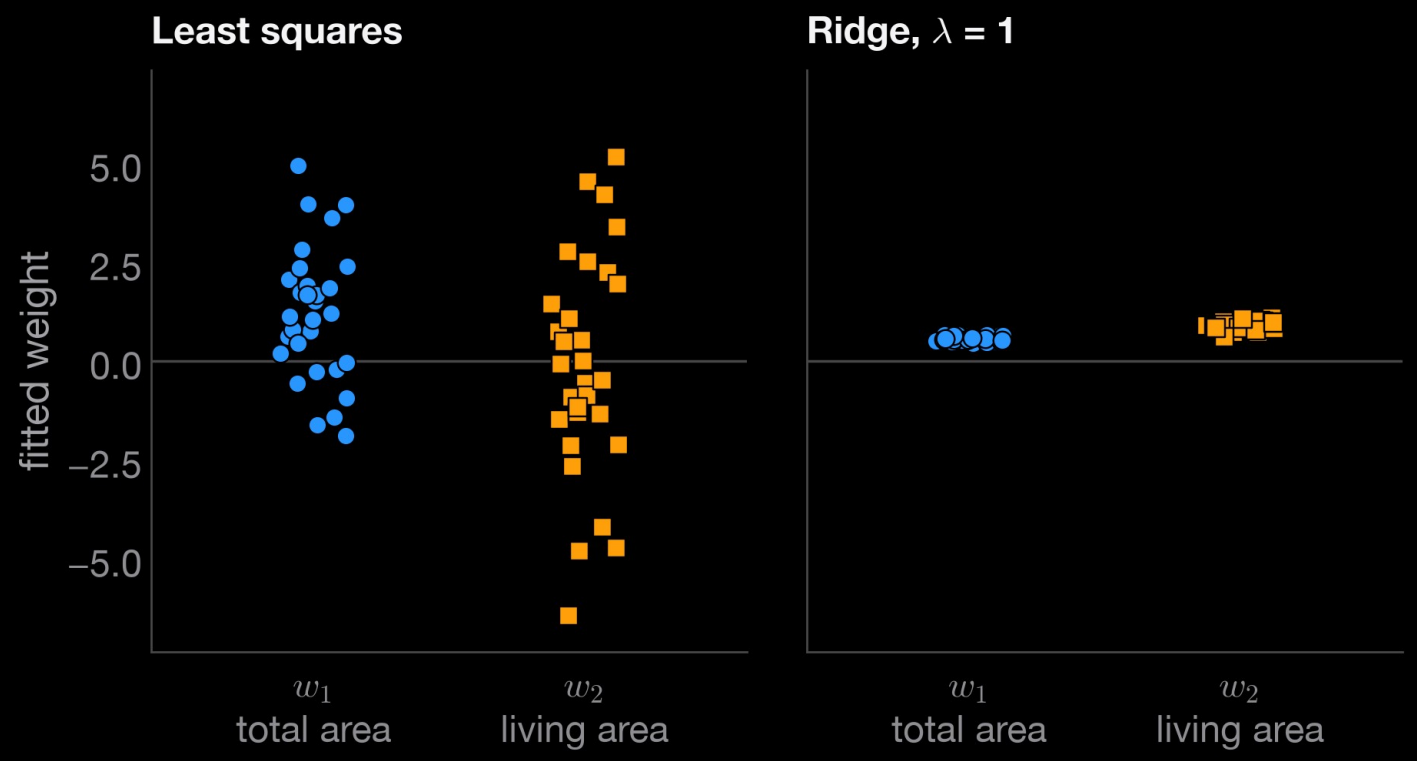
The variance is taken along one direction of the data.

ridge

$$\mathbb{E}\,\hat{w} = w - \lambda\,(X^\top X + \lambda I)^{-1} w, \qquad \mathrm{Var}\left(v_j^\top \hat{w}\right) = \frac{\sigma^2 d_j^2}{(d_j^2 + \lambda)^2} \leq \frac{\sigma^2}{4\lambda}$$

Least squares is unbiased, and its variance blows up along a weak direction.
Ridge accepts a bias and gets a variance that stays bounded.

The twins stop swinging



Ridge shares the weight between the twins.

Least squares: from –6.4 to 5.2. Ridge: from 0.5 to 1.1.

The estimate is biased now, and far more stable.

The listing: ten columns, four matter

Forty flats in Tashkent. The price depends on four of the columns.

area	living	rooms	floor	centre	metro	age	day	house	photos	price
77	44.4	2	12	3.4	3.5	12	1	42	15	115.3
104	61.1	4	5	13	3.1	7	20	37	19	103.6
106	62.1	4	12	8.7	1.8	25	25	63	14	134.9
70	42.7	2	8	10.4	3.2	57	19	109	18	70.9
101	62.6	4	12	7.9	1.3	17	20	27	25	123.8

living and rooms repeat the area. floor, day, house and photos do nothing.

Least squares on the listing

Ten weights from forty flats.

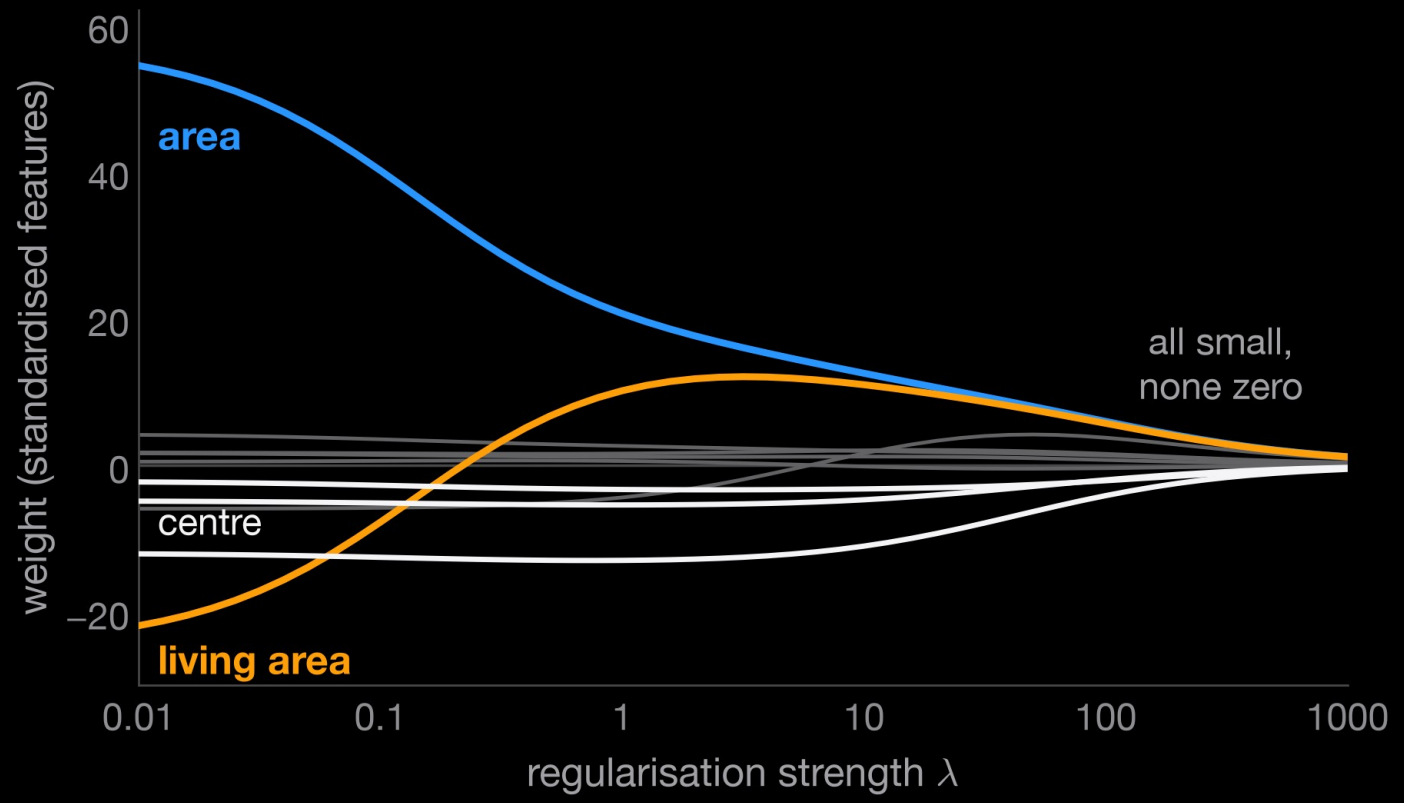
feature	truth	least squares
area	1.1	**2.43**
living	0	**–1.72**
rooms	0	**–5.55**
centre	–3.0	–2.96
metro	–4.0	–4.15
age	–0.3	–0.13

10.6 RMSE on its 40 flats

14.5 RMSE on 400 new flats

Thousand dollars. The noise level is 12.

Ridge shrinks every weight



The twins meet and share.

At λ = 2.5 the RMSE on new flats falls from 14.5 to 13.4.

No weight reaches zero. Ten features stay ten.

Lasso: the L1 penalty

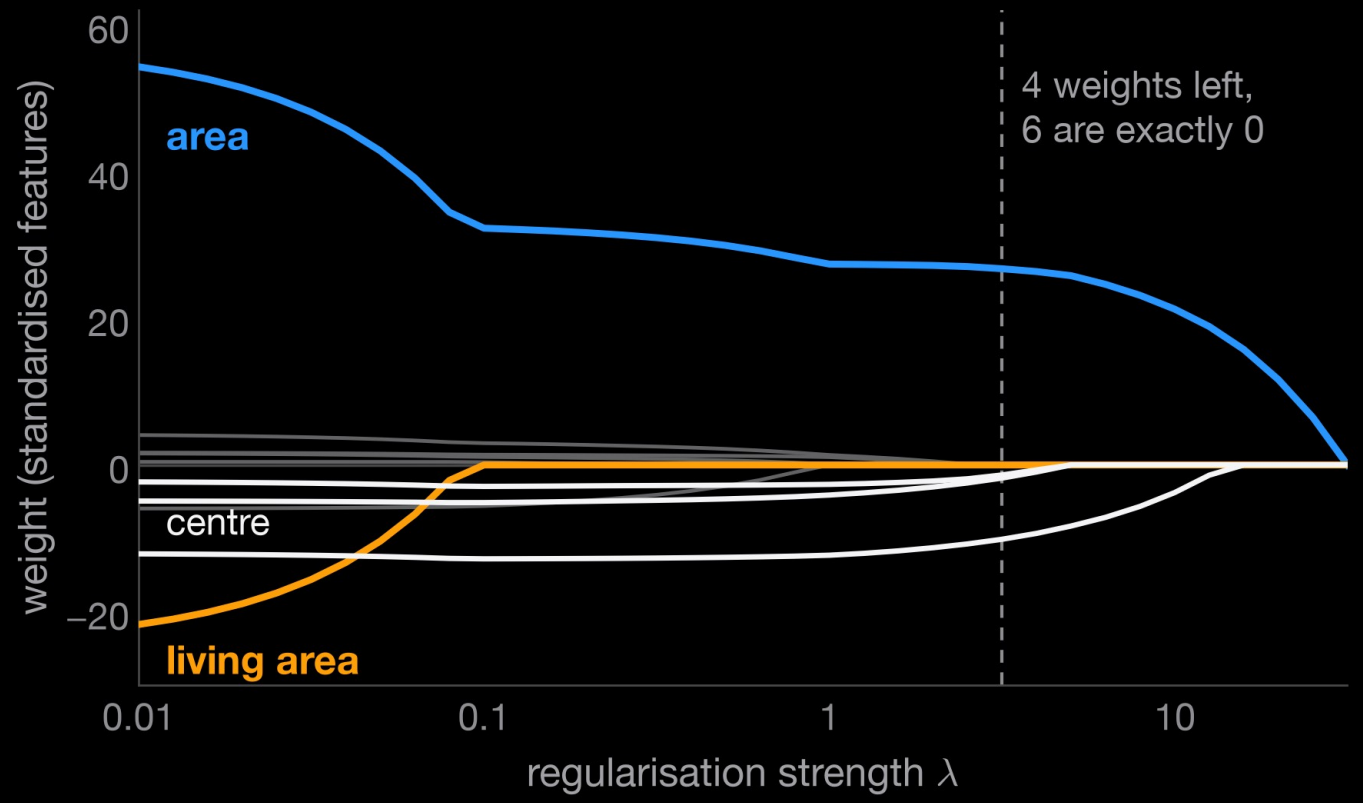
$$L(w) = \|y - Xw\|^2 + \lambda \|w\|_1$$

$$\|w\|_1 = \sum_{j=1}^{p} |w_j|$$ the sum of absolute weights

No closed formula. The loss is convex, and coordinate descent solves it.

Tibshirani, Regression shrinkage and selection via the lasso, 1996

Lasso sets weights to zero



Weights reach exactly zero, one after another.

At λ = 3.2 four are left: area, centre, metro and age, the four that set the price.

Lasso fits the model and selects the features in one step.

Three fits of the same listing

feature	truth	least squares	ridge, λ = 2.5	lasso, λ = 3.2
area	1.1	2.43	0.71	**1.14**
living	0	–1.72	0.84	0
rooms	0	–5.55	–2.42	0
floor	0	0.50	0.34	0
centre	–3.0	–2.96	–3.13	**–2.49**
metro	–4.0	–4.15	–4.56	**–1.52**
age	–0.3	–0.13	–0.19	**–0.08**
day	0	0.22	0.22	0
house	0	0.10	0.05	0
photos	0	0.06	0.05	0
RMSE on new flats		**14.5**	**13.4**	**13.6**

The same problem with a budget

Two ways to write one problem.

$$\min_w \|y - Xw\|^2 + \lambda R(w)$$

a price on the penalty

$$\min_w \|y - Xw\|^2 \quad \text{subject to} \quad R(w) \leq t$$

a budget for the penalty

For every λ there is a budget t with the same solution. A larger λ means a smaller budget.

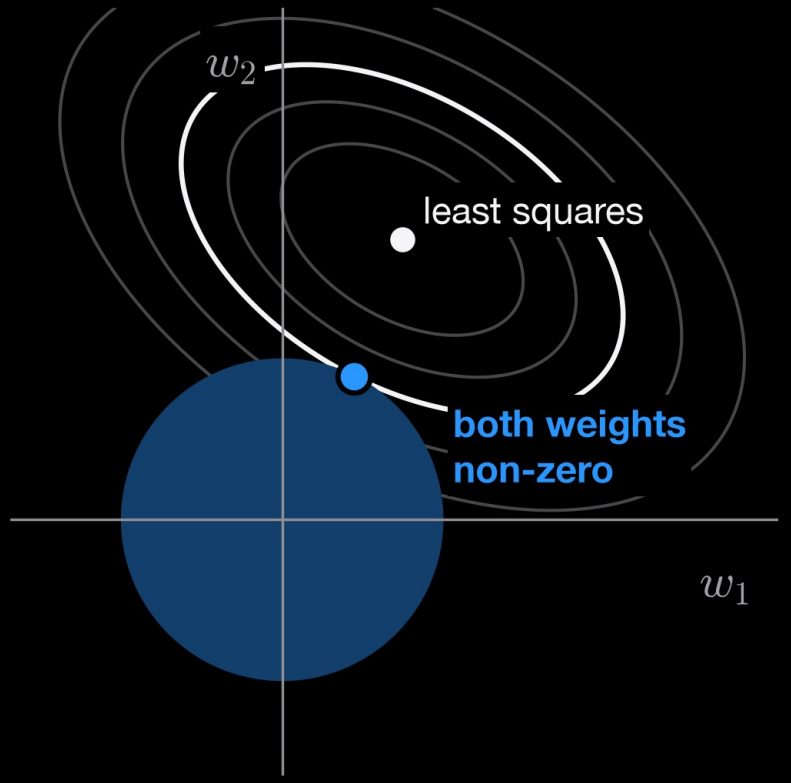
$\|w\|_2^2 \leq t$ a disc

$\|w\|_1 \leq t$ a diamond

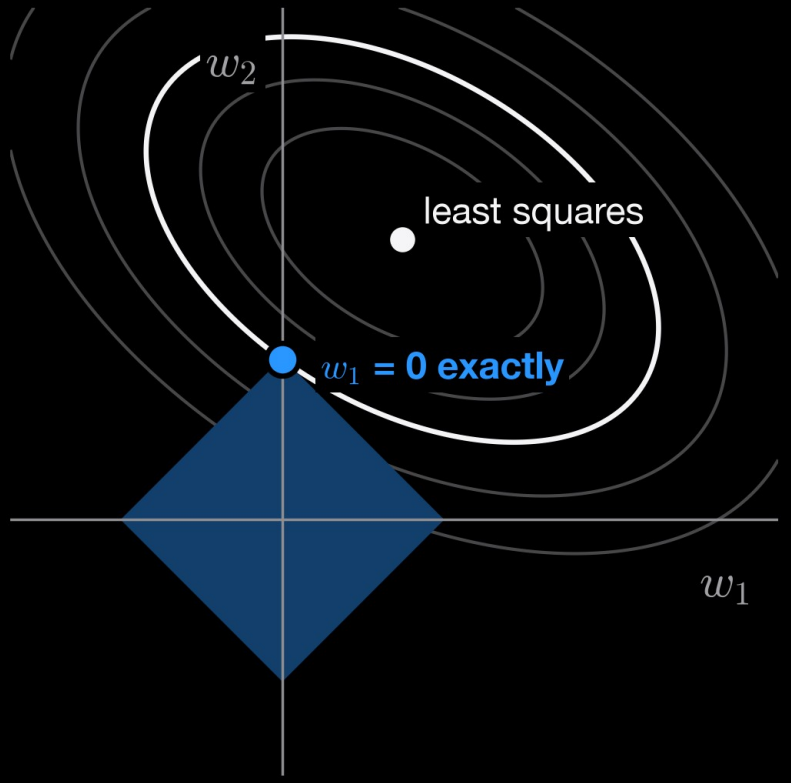
Why lasso gives zeros

The solution is where a loss contour first touches the penalty region.

Ridge: a disc

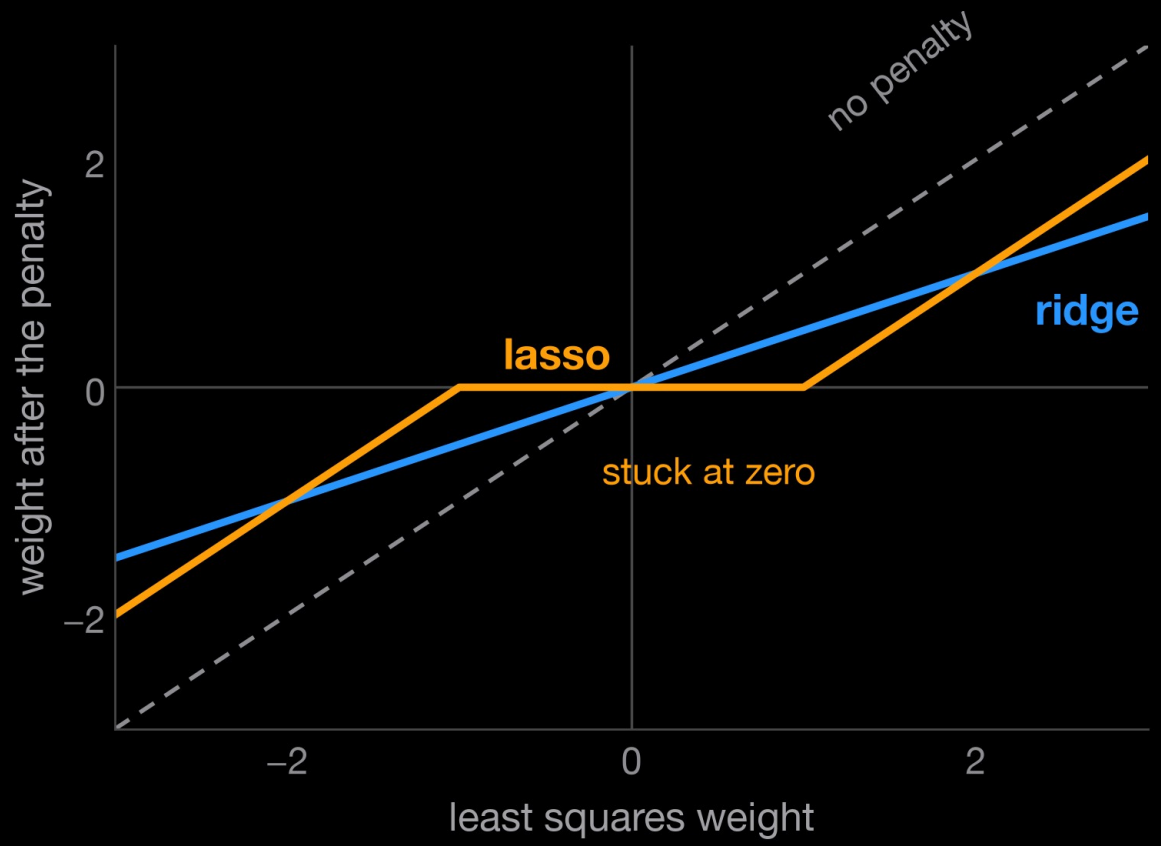


Lasso: a diamond



One weight at a time

For uncorrelated features each penalty acts on every weight separately.



$$\hat{w}_j = \frac{w_j^{\mathrm{LS}}}{1+\lambda}$$

Ridge divides. Large weights lose the most, none reaches zero.

$$\hat{w}_j = \mathrm{sign}\left(w_j^{\mathrm{LS}}\right)\ \max\left(\left|w_j^{\mathrm{LS}}\right| - \frac{\lambda}{2},\ 0\right)$$

Lasso subtracts the same amount from each and stops at zero.

Where the two formulas come from

Take orthonormal features: the columns of X have unit length and are uncorrelated.

$$X^\top X = I \quad \Longrightarrow \quad w^{\mathrm{LS}} = X^\top y$$

$$\|y - Xw\|^2 = \underbrace{\|y - Xw^{\mathrm{LS}}\|^2}_{\text{does not depend on } w} + \sum_{j=1}^{p} \left(w_j - w_j^{\mathrm{LS}}\right)^2$$

The loss is a sum over the weights, so each weight is found on its own.

$$\min_{w_j} \left(w_j - w_j^{\mathrm{LS}}\right)^2 + \lambda\, w_j^2 \quad \Longrightarrow \quad 2\left(w_j - w_j^{\mathrm{LS}}\right) + 2\lambda w_j = 0 \quad \Longrightarrow \quad \hat{w}_j = \frac{w_j^{\mathrm{LS}}}{1+\lambda}$$

For ridge the equation is linear. The lasso takes one more slide.

Lasso: the soft threshold

One weight. Write *a* for its least squares value.

$$g(w) = (w - a)^2 + \lambda\,|w|$$

$$w > 0: \quad g'(w) = 2(w-a) + \lambda = 0 \quad \Longrightarrow \quad w = a - \tfrac{\lambda}{2}, \quad \text{if } a > \tfrac{\lambda}{2}$$

$$w < 0: \quad g'(w) = 2(w-a) - \lambda = 0 \quad \Longrightarrow \quad w = a + \tfrac{\lambda}{2}, \quad \text{if } a < -\tfrac{\lambda}{2}$$

$$|a| \le \tfrac{\lambda}{2}: \quad \text{neither case is possible} \quad \Longrightarrow \quad w = 0$$

$$\hat{w} = \operatorname{sign}(a)\,\max\left(|a| - \frac{\lambda}{2},\, 0\right)$$

The absolute value has a kink at zero. A whole interval of least squares weights lands on it, and that is where the exact zeros come from.

Any features: coordinate descent

Fix all the weights but one. For that one the problem is the soft threshold again.

$$r^{(j)} = y - \sum_{k \neq j} x_k w_k$$

the part of *y* that the other features leave unexplained

$$w_j \leftarrow \frac{S_{\lambda/2}\left(x_j^\top r^{(j)}\right)}{\|x_j\|^2}$$

$$S_c(a) = \text{sign}(a)\ \max\big(|a| - c,\ 0\big)$$

Sweep over the weights until they stop changing. The loss is convex, so the sweeps reach its minimum.

scikit-learn fits Lasso and ElasticNet this way.

Friedman, Hastie, Tibshirani, Regularization paths for generalized linear models via coordinate descent, 2010

Elastic net: both penalties

$$L(w) = \|y - Xw\|^2 + \lambda_1 \|w\|_1 + \lambda_2 \|w\|_2^2$$

Ridge

Keeps every feature. Stable when features are correlated.

Lasso

Selects features. Of two twins it keeps one, and which one is arbitrary.

Elastic net

Selects features and keeps correlated ones together.

Zou and Hastie, Regularization and variable selection via the elastic net, 2005

The intercept is not penalised

$$\lambda \sum_{j=1}^{p} w_j^2$$

The sum starts at 1.

The intercept stays free. It carries the average price, and shrinking it would pull every prediction towards zero.

Standardise before you penalise

$$x_j \leftarrow \frac{x_j - \bar{x}_j}{s_j}$$

The penalty compares weights, so the features must share a scale.

Area in m² needs a weight near 1. The same area in cm² needs 0.0001, and the penalty stops noticing it.

Fit the scaler on the training set only.

Where the penalty comes from

Last week the weights were unknown constants. Now they get a prior.

$$p(w \mid X, y) \;\propto\; p(y \mid X, w)\, p(w)$$

$$\hat{w}_{\mathrm{MAP}} = \arg\max_{w} \left[\log p(y \mid X, w) + \log p(w) \right]$$

Maximum a posteriori: the most probable weights given the data and the prior.

From the prior to the penalty

Normal noise, and a normal prior on every weight.

$$\log p(y \mid X, w) = -\frac{1}{2\sigma^2} \|y - Xw\|^2 + \text{const}$$

$$\log p(w) = -\frac{1}{2\tau^2} \|w\|_2^2 + \text{const}$$

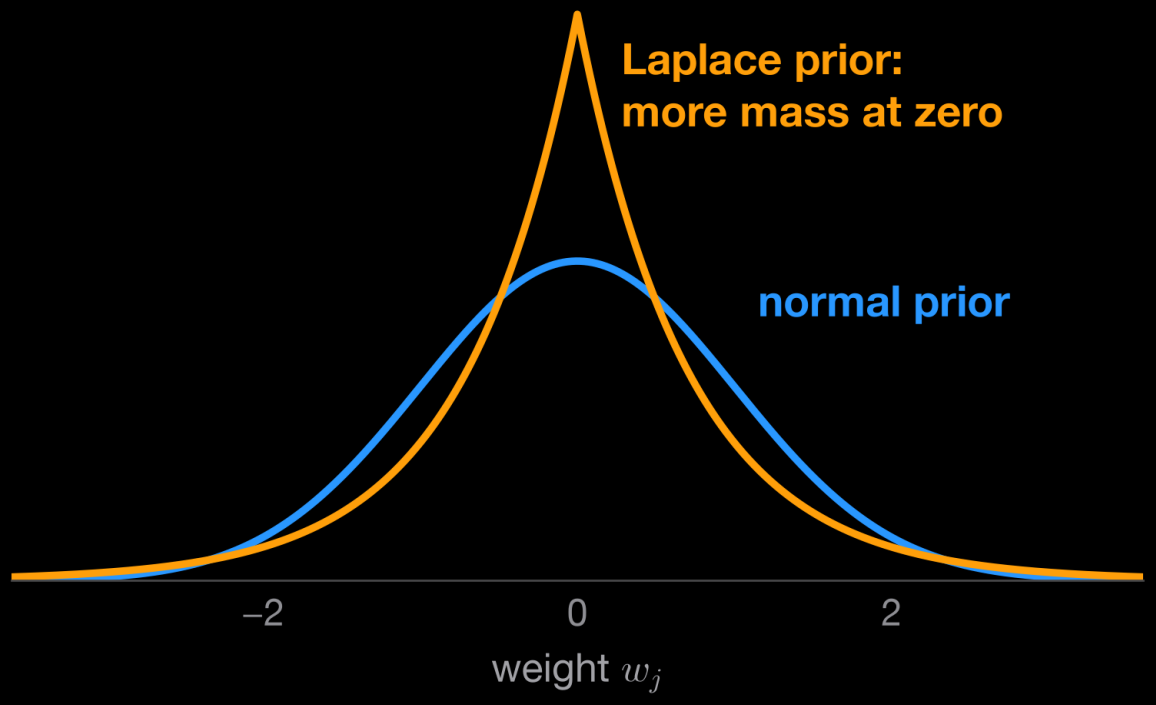
$$\hat{w}_{\text{MAP}} = \arg\min_w \left[\|y - Xw\|^2 + \frac{\sigma^2}{\tau^2} \|w\|_2^2 \right]$$

Add the two lines and change the sign: the result is the ridge loss.

$$\log p(w) = -\frac{1}{b} \|w\|_1 + \text{const} \quad \Longrightarrow \quad \lambda = \frac{2\sigma^2}{b}$$

the same steps with a Laplace prior

A prior on the weights is a penalty



$$w_j \sim \mathcal{N}(0, \tau^2) \quad \Longrightarrow \quad \lambda \|w\|_2^2$$

$$w_j \sim \text{Laplace}(0, b) \quad \Longrightarrow \quad \lambda \|w\|_1$$

$$\lambda = \frac{\sigma^2}{\tau^2}$$ for ridge with normal noise

A tight prior or noisy data: a large λ.

Normal noise gives squared error. A normal prior gives the L2 penalty.

Laplace noise gave absolute error last week. A Laplace prior gives the L1 penalty.

The same idea in neural networks

$$w \leftarrow (1 - 2\eta\lambda)\, w - \eta \nabla L(w)$$

Each step shrinks every weight a little, then follows the gradient. Deep learning calls it weight decay.

Early stopping and dropout are regularisers too. They come later in the course.

Regularisation in scikit-learn

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge, Lasso

ridge = make_pipeline(StandardScaler(), Ridge(alpha=2.5))
lasso = make_pipeline(StandardScaler(), Lasso(alpha=3.2))
lasso.fit(X_train, y_train)
lasso[-1].coef_          # six of the ten weights are 0.0
```

scikit-learn calls λ alpha. Ridge and Lasso scale the squared error differently, so their alphas cannot be compared.

Hyperparameters and validation

Parameters and hyperparameters

Parameters

Learned from the training data.

The weights of the model.

Found by the normal equations or gradient descent.

Hyperparameters

Set before training starts.

The degree, λ, k in kNN, the learning rate.

Found by trying values and comparing the models.

Training error cannot choose

It always votes for the most flexible model.

degree 15

The degree with the lowest training error.

$\lambda = 0$

The λ with the lowest training error.

A test set used to choose a model is no longer a test set.

Choose λ on it, and the error you report is the best of many tries.

Three sets, three jobs

The split from week 1. Each part answers one question.



Fit the weights.

Least squares, gradient descent.

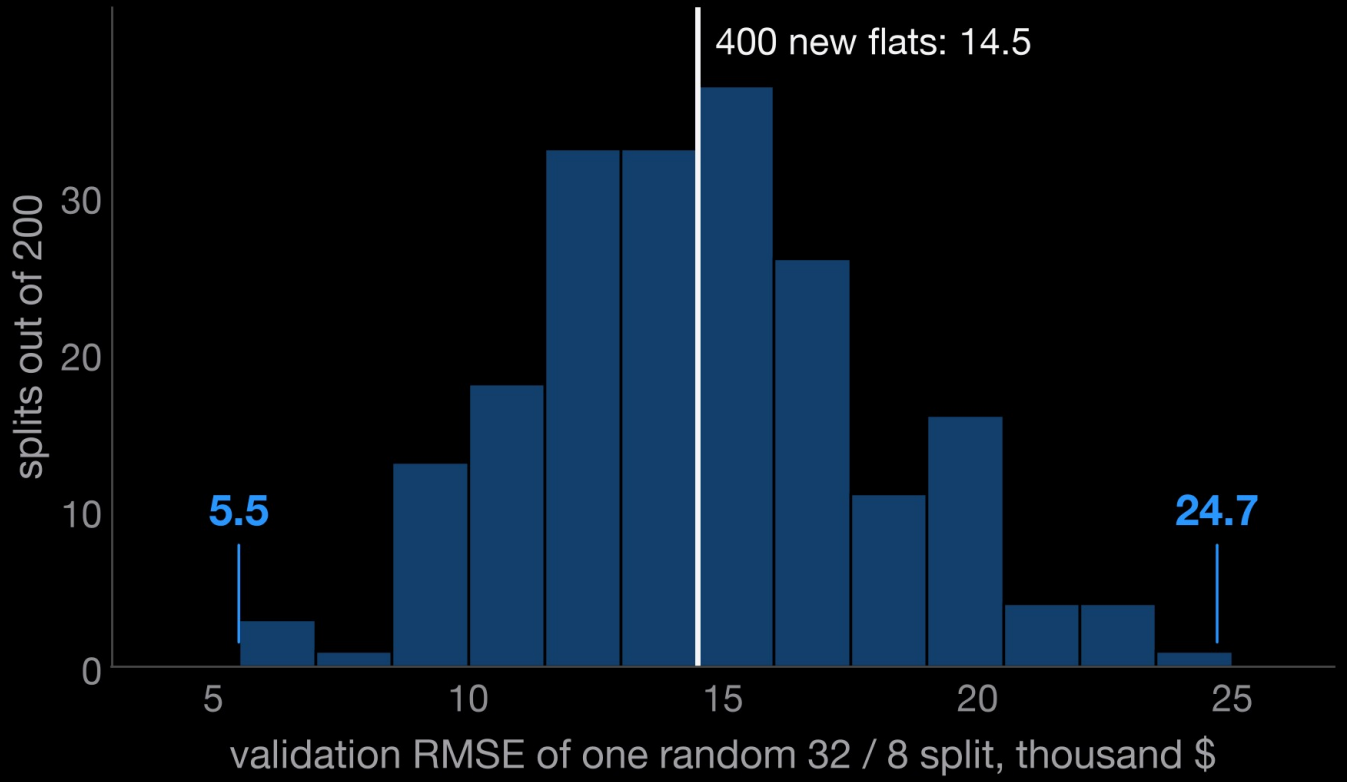
Choose λ.

And every other hyperparameter.

Report.

One number, once.

One split of 40 flats is a lottery

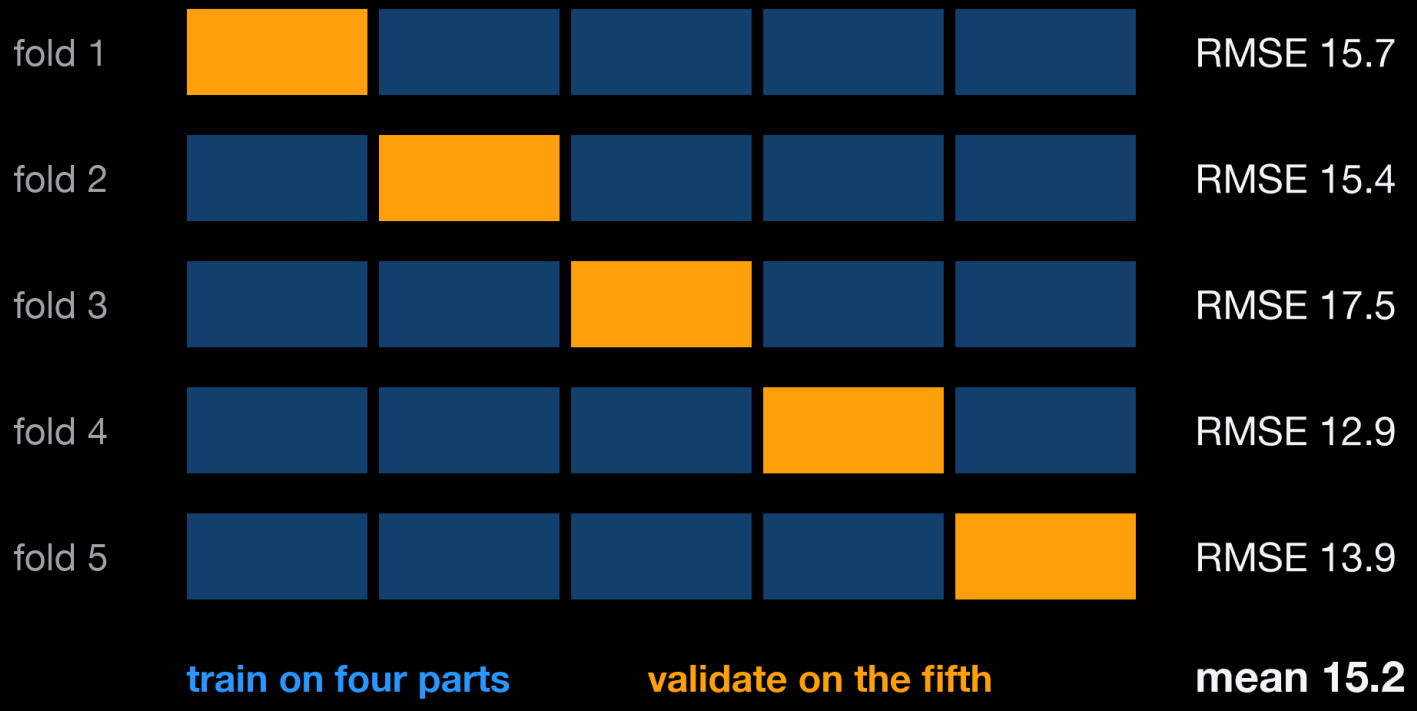


5.5 to 24.7

The validation RMSE of the same model over 200 random splits.

With 8 flats in validation the number depends on which 8.

k-fold cross-validation

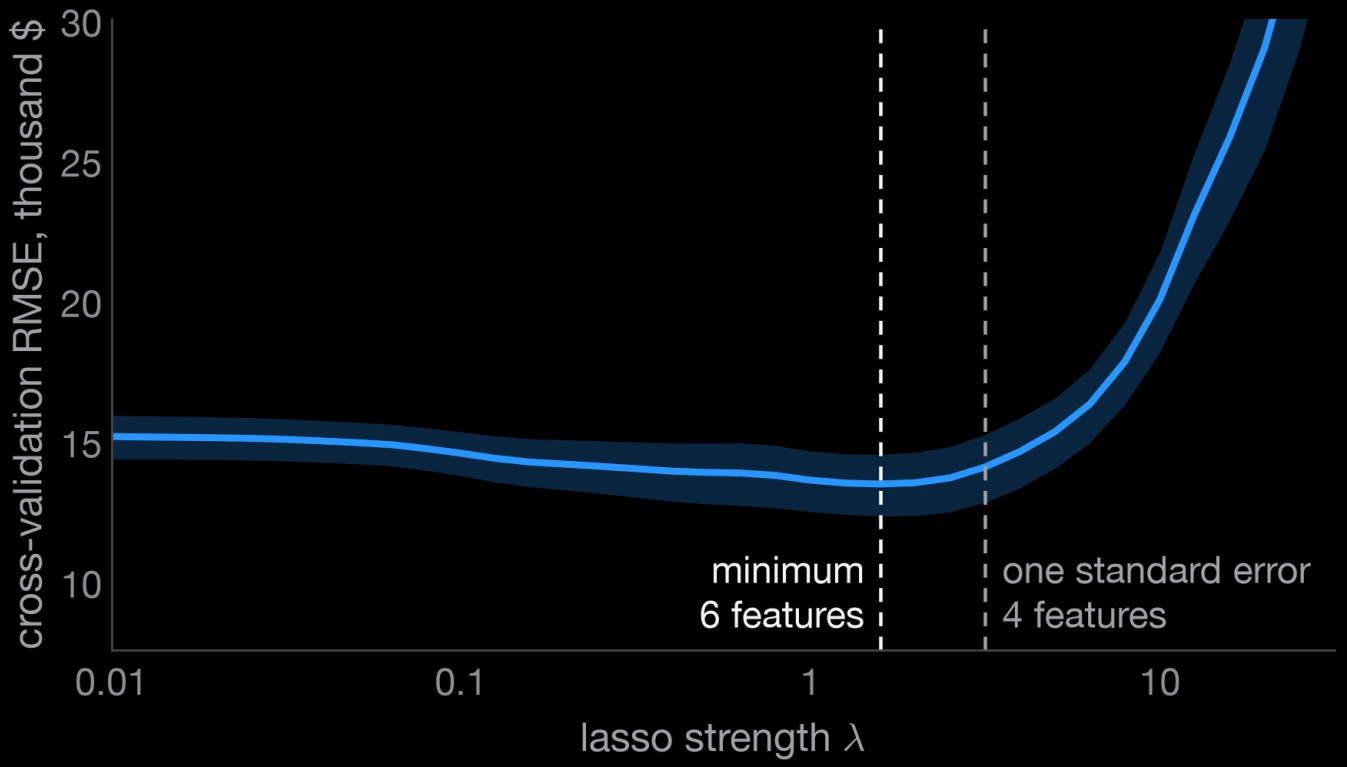


Every flat is validated exactly once.

Cut the data into k parts. Train on k – 1, validate on the last one, rotate.

Average the k errors.

Choosing λ by cross-validation

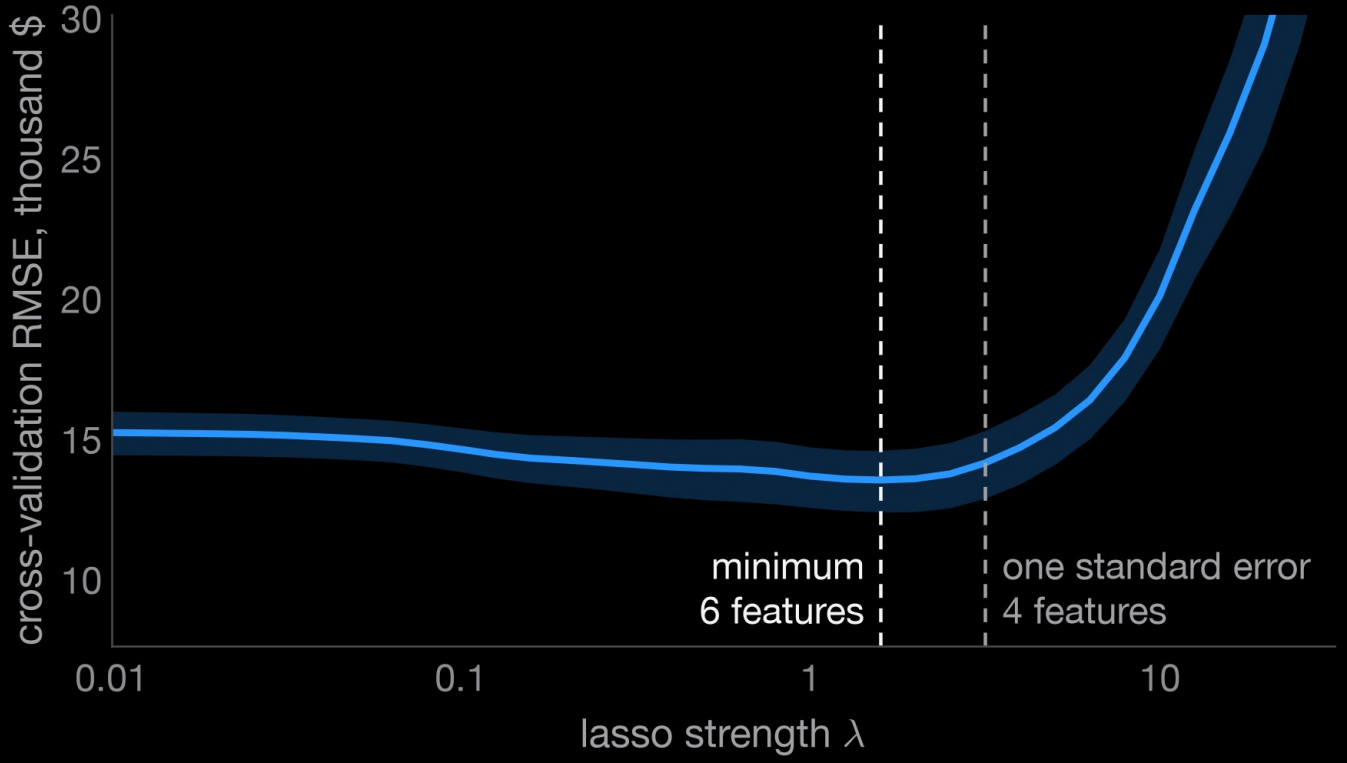


$$\mathrm{CV}(\lambda) = \frac{1}{k}\sum_{f=1}^{k} \mathrm{MSE}_f(\lambda)$$

$$\hat{\lambda} = \arg\min_{\lambda} \ \mathrm{CV}(\lambda)$$

Lasso on the listing: the minimum is at λ = 1.6, with 6 features.

The one standard error rule



Take the simplest model within one standard error of the best.

The curve is flat near its minimum and the folds are noisy.

$\lambda = 3.2$ is as good within the noise and keeps only the 4 real features.

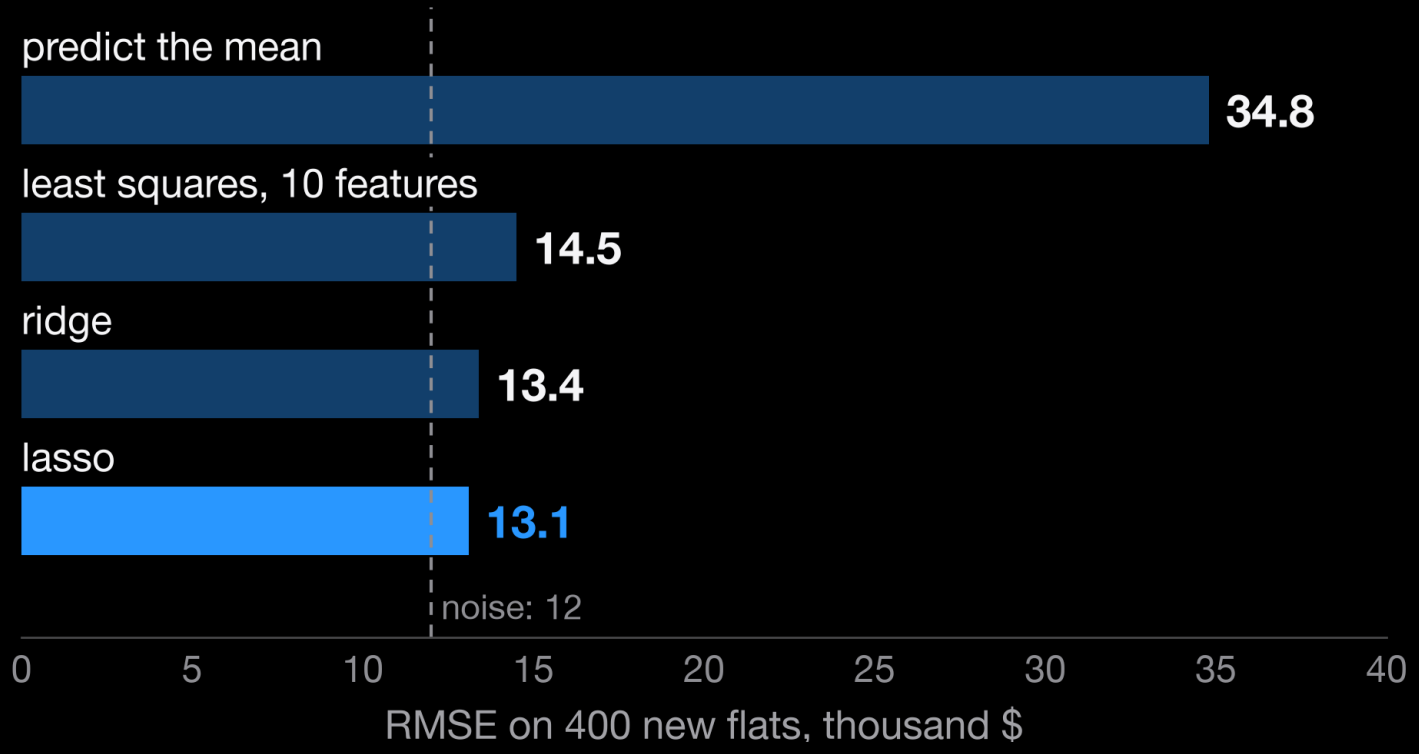
The scaler goes inside the folds

```
from sklearn.model_selection import GridSearchCV, KFold

model = make_pipeline(StandardScaler(), Lasso())
grid = {"lasso__alpha": np.logspace(-2, 1.5, 36)}
folds = KFold(5, shuffle=True, random_state=0)
search = GridSearchCV(model, grid, cv=folds,
                      scoring="neg_mean_squared_error")
search.fit(X_train, y_train)
search.best_params_      # {'lasso__alpha': 1.58}
```

The pipeline refits the scaler on each training fold. Scaling all the data first would leak the validation folds into training.

Then open the test set, once



Refit on all 40 flats with the chosen λ. Then test.

Regularisation took the error on new flats from 14.5 to 13.1.

The noise alone is 12.

Cross-validation or one fixed split

Cross-validation

Small data, up to tens of thousands of rows.

Every row is used for training and for validation.

Costs k trainings per candidate.

One fixed split

Large data: the validation set is big enough to trust.

One training per candidate.

The usual choice in deep learning.

Splitting the data right

The validation set must look like the data the model will meet.

If validation is easier than real use, its number is better than the real one.

Random split



Shuffle, then cut.

Right when the objects are independent: one flat tells nothing about another. This is what train_test_split does by default.

Stratified split



Keep the class shares in every part.

One mark in four is a square, in training and in validation alike. Needed for classification with rare classes. In scikit-learn: stratify=y, StratifiedKFold.

Grouped split

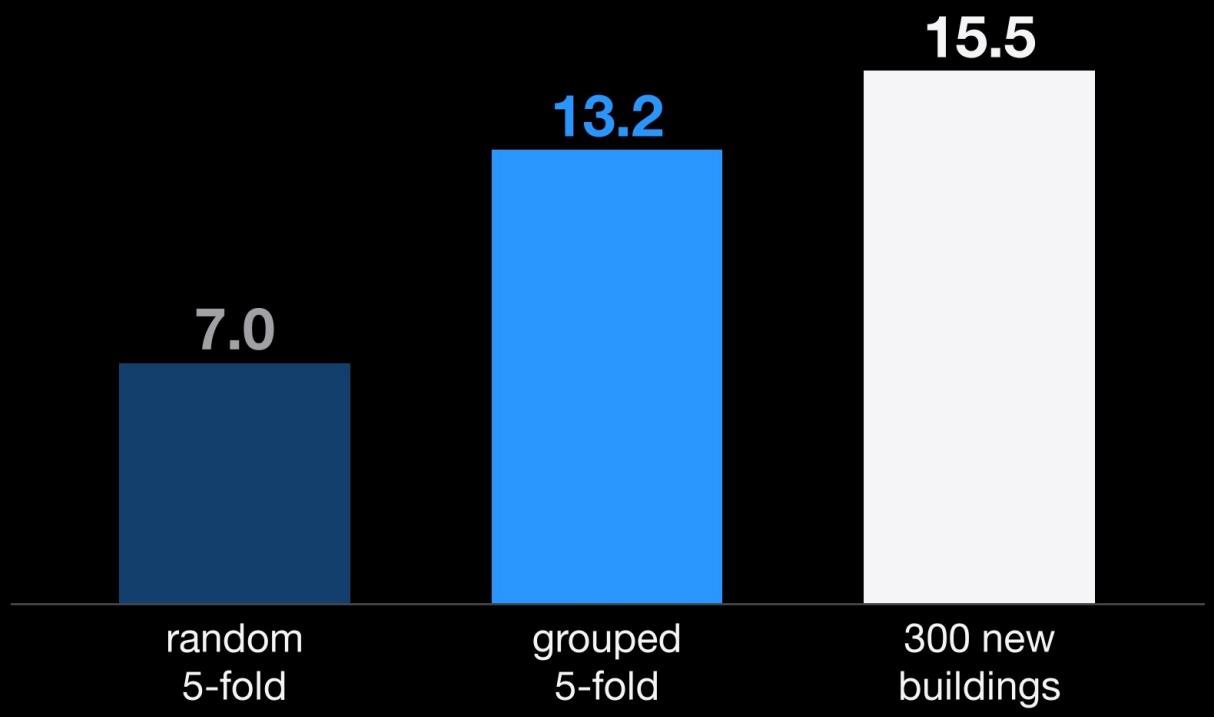


A whole group goes to one side.

Flats of one building, visits of one patient, queries of one user. In scikit-learn: GroupKFold.

What a random split hides

30 buildings, 4 flats in each. The model knows the building of every flat.

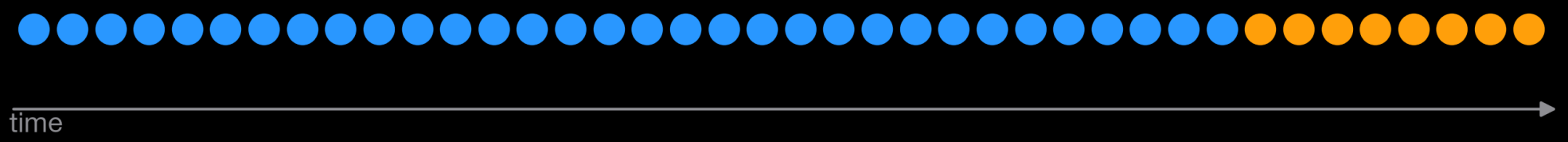


Random folds promise 7.0.

The model has already seen the building of every validation flat. On 300 new buildings the error is 15.5.

Grouped folds say 13.2: close to the truth, and known before deployment.

Time split



Train on the past, validate on the future.

Prices, demand, anything with a date. A random split lets the model see tomorrow. In scikit-learn: TimeSeriesSplit.

Which split?

Ask how the model will be used, and split the same way.

split	use it when	scikit-learn
Random	the objects are independent	KFold(shuffle=True)
Stratified	classes, some of them rare	StratifiedKFold
Grouped	objects come in groups	GroupKFold
Time	the model predicts the future	TimeSeriesSplit

Revise

More capacity lowers the training error. The error on new data is U-shaped.

Ridge shrinks every weight. Lasso sets some of them exactly to zero.

Hyperparameters are chosen on validation data. The test set is opened once.

Split the data the way the model will be used: by group, by time.

Exit quiz

Scan the code.

Sign in. One question at a time, 30 seconds each.

If time allows, we then play the multiplayer game Neighbours.



165.22.93.84

Practice

Seminar

One notebook: overfitting, ridge and lasso, cross-validation.

Interactive

Six pages at 165.22.93.84/viz: overfitting, ridge, lasso, cross-validation, splits that leak.

Self-check

Practice questions with explanations, on the same site.

Links to the notebook are in the course chat.

To read

Books

Hastie, Tibshirani, Friedman, The Elements of Statistical Learning: 3.4 and 7.10

Bishop, Pattern Recognition and Machine Learning: 1.1, 3.1.4 and 3.3

James, Witten, Hastie, Tibshirani, An Introduction to Statistical Learning: chapters 5 and 6

Online

scikit-learn user guide: linear models, cross-validation

StatQuest on YouTube: Ridge, Lasso and Elastic Net regression

Yandex ML handbook, in Russian: linear models, cross-validation

Next time: logistic regression

A linear model that predicts a class. Bring your questions about the seminar notebook.

Questions?

Makhmudjon Sodikov, YMLS Uzbekistan

YMLS



girafe ai